

Python 语法手册

—— 配 ServerTimeDemo 项目实例

下册

工程实战与附录 · 第十一章 ~ 第十三章 + 附录 A ~ D

从词法到工程约定，逐条对照真实代码

74 个 Python 文件 · 44,490 行 · 23 张数据表

项目版本 1.9.0

手册版本 1.0

生成日期 2026 年 09 月 21 日

《Python 语法手册》全三册 · 本册为下册

本手册的每一段示例代码都摘自本项目源码，未作伪写

目录

本册是三册中的**下册**，收录 第十一章 ~ 第十三章 + 附录 A ~ D。（工程实战与附录）

第十一章 标准库实战	4
11.1 pathlib：路径就用 / 拼	4
11.2 datetime 与 calendar：日期计算的三个必备套路	4
11.3 json：结构化数据的通用交换格式	6
11.4 sqlite3：本项目唯一的持久化方案	7
11.5 logging：一份日志基建的最小正确实现	11
11.6 re：正则的四个实用要点	12
11.7 其它常用模块	13
第十二章 GUI 相关的 Python 语法	15
12.1 界面为什么用类组织	15
12.2 回调：传函数对象，不传调用结果	15
12.3 after：Tkinter 的定时器	16
12.4 事件绑定与事件对象	18
12.5 变量类：StringVar / IntVar / BooleanVar	18
12.6 控件生命周期与 TclError	19
12.7 一个跨语言的坑：Tk 会「饿死」控件	21
第十三章 工程约定与陷阱	22
13.1 行尾：CRLF 与 LF	22
13.2 文本 vs 字节：什么时候必须用字节	23
13.3 编码：统一 UTF-8，一处都不要例外	23
13.4 时间：本项目明确选了「本机时间」	24
13.5 数值：先把误差摆到台面上	25
13.6 静态检查：pyflakes 与它的口径	25
13.7 打包：Python 特有的三个认知	26
附录 A • 语法速查卡	27
A.1 数据类型与字面量	27
A.2 字符串与容器操作	27
A.3 控制流与函数	27

A.4 类	28
A.5 模块与导入	28
A.6 常用标准库	29
A.7 高频惯用法	29
附录 B · 模块清单	31
B.1 数据目录结构	34
B.2 测试套件与基线	34
附录 C · 坑对照表	36
C.1 语法与语义类	36
C.2 文件与编码类	36
C.3 GUI 类	37
C.4 数据与打包类	38
附录 D · 工具脚本与开发流程	39
D.1 这本手册自己怎么生成	39
D.2 本项目自己的「补丁纪律」	40
D.3 这套流程的通用形状	40
D.4 待办（项目的下一步）	41
结语	41

第十一章 标准库实战

Python 的「自带电池」在这项目里用得很实际：**没有引入任何 Web 框架、ORM、配置库**，全部靠标准库完成。这一章按模块讲，每个都配项目里的真实用法。

11.1 pathlib：路径就用 / 拼

```
1 from pathlib import Path
2
3 BASE_DIR = Path(__file__).resolve().parent
4 DATA_DIR = BASE_DIR / "ExpiryManager_Data"
5 LOG_DIR = DATA_DIR / "logs"
6 DB_PATH = DATA_DIR / "expiry_manager.db"
7
8 LOG_DIR.mkdir(parents=True, exist_ok=True)      # 不存在就建，存在也不报错
```

▲ main.py · 路径拼接与目录创建

操作	写法
拼接	<code>p / "sub" / "file.txt"</code> (/ 被重载成拼接)
取父目录	<code>p.parent</code> 、 <code>p.parents[1]</code>
取文件名 / 后缀 / 主干	<code>p.name</code> 、 <code>p.suffix</code> 、 <code>p.stem</code>
绝对化	<code>p.resolve()</code>
存在性	<code>p.exists()</code> 、 <code>p.is_file()</code> 、 <code>p.is_dir()</code>
建目录	<code>p.mkdir(parents=True, exist_ok=True)</code>
遍历	<code>p.iterdir()</code> 、 <code>p.glob("*.py")</code> 、 <code>p.rglob("*.py")</code>
读 / 写文本	<code>p.read_text(encoding="utf-8")</code> / <code>p.write_text(...)</code>
读 / 写字节	<code>p.read_bytes()</code> / <code>p.write_bytes(...)</code>

`write_text` 会把行尾换成 CRLF (Windows)

`Path.write_text()` 走的是文本模式，Windows 下默认把 `\n` 翻译成 `\r\n`。本项目因此踩过一次：一个纯 LF 的测试脚本被 `write_text` 写回后变成 946 个 CRLF，`git diff` 从「改几行」炸成「改 946 行」。要精确保住行尾，只能用 `write_bytes()`。详见 13.1、13.2。

11.2 datetime 与 calendar：日期计算的三个必备套路

套路一：按自然月推进，天数不足时收敛到月末

```
1 def add_months(base: date, months: int) -> date:
2     """按自然月推进，遇到目标月天数不足时收敛到该月最后一天。
3
4     例：1 月 31 日 + 1 个月 → 2 月 28/29 日（而不是抛异常或滚到 3 月）。
5     """
6     total = (base.year * 12 + base.month - 1) + months
7     year, month = divmod(total, 12)
8     month += 1
9     day = min(base.day, calendar.monthrange(year, month)[1])
10    return date(year, month, day)
11
12
13 def add_years(base: date, years: int) -> date:
14     """按自然年推进，2 月 29 日在平年收敛到 2 月 28 日。"""
15     year = base.year + years
16     day = min(base.day, calendar.monthrange(year, base.month)[1])
17     return date(year, base.month, day)
```

▲ todo_db.py · 重复规则里「每月 / 每年」的推进逻辑

这里的三个知识点

- ① `divmod(总月数, 12)` 一次同时拿到年和月 —— 把「年 + 月」压成一个线性坐标再除，比手工处理进位干净得多。
- ② `calendar.monthrange(y, m)[1]` 拿到该月天数，免去自己写闰年判断。
- ③ `min(base.day, 月末)` 是「收敛」而非「报错」：1 月 31 日加一个月落到 2 月 28/29 日，而不是抛 `ValueError`。**日期推进的默认策略应该是收敛** —— 用户设了「每月 31 日」，到 2 月时他期待的是「2 月最后一天」，不是一个报错。

套路二：格式化与解析要配对

```
1 def moment_str(value: datetime) -> str:
2     """datetime → ``YYYY-MM-DD HH:MM``（提醒精确到分钟，秒没有意义）。"""
3     return value.strftime("%Y-%m-%d %H:%M")
4
5
6 def _now() -> str:
7     """当前时间的 ISO 字符串（精确到秒）。"""
8     return datetime.now().isoformat(timespec="seconds")
```

▲ todo_db.py · 项目里时间入库只走这两个函数

格式码	含义	易错点
%Y-%m-%d	2026-09-21	大写的 %Y 是四位年
%H:%M	14:05	24 小时制用大写 %H; %I 是 12 小时制
%S	秒	
%f	微秒	6 位

格式码	含义	易错点
%j	一年中的第几天	
%a / %A	星期缩写 / 全称	受系统语言影响，别用来判断星期

中文 Windows 上 %a 出来是中文星期

`strftime("%a")` 依赖系统 locale，在中文系统上可能是「周一」。要判断星期几一律用 `date.weekday()` (**0 = 周一**) 或 `isoweekday()` (**1 = 周一**)，不要解析格式化的字符串。本项目判断「是不是周末」用的就是数值。

套路三：比较与差值

```

1 now = now or datetime.now()
2 base = alert_moment(item) or now
3 nxt = max(base, now) + timedelta(minutes=max(1, minutes))

```

▲ `todo_db.py · snooze()`: `max(datetime, datetime)` 也是一行

`datetime` 支持比较、支持加减 `timedelta`，所以「两个时间取较晚的那个」直接用内建 `max` 就行，不需要写 `if`。

`datetime.now()` 是「本机时间」，没有时区信息

它返回的是 naive datetime (`tzinfo` 为 `None`)，含义是「这台机器认为的本地时间」。本项目是**单机自用工具**，所有时间都取自本机、也只在本机比较，所以 naive 时间没有问题。

但只要出现「跨机器同步」「服务器时间」「夏令时」任何一项，就必须换成 `datetime.now(timezone.utc)` 存 UTC、展示时再 `astimezone()` 到本地——否则一定会在某个时刻错一小时。

11.3 json：结构化数据的通用交换格式

```

1 @classmethod
2 def from_row(cls, row: sqlite3.Row) -> "TodoItem":
3     raw = row["tags"]
4     if isinstance(raw, str):
5         try:
6             tags = json.loads(raw)
7         except Exception:
8             tags = [t.strip() for t in raw.split(",") if t.strip()]
9     elif isinstance(raw, list):
10         tags = raw
11     else:
12         tags = []

```

▲ `todo_db.py · 从数据库读 JSON 列`，带三级降级

这段是本项目里「**宽容解析**」的范本：先按 JSON 解；解不出来就退化成逗号分隔的纯文本；连类型都不对（既不是字符串也不是列表）就当空。**老旧数据、手工改过的数据都能读进来**，不会因为一行脏数据让整个列表加载失败。

函数	用途
<code>json.dumps(obj, ensure_ascii=False, indent=2)</code>	转字符串； 中文要 <code>ensure_ascii=False</code>
<code>json.loads(text)</code>	解析字符串
<code>json.dump(obj, f) / json.load(f)</code>	直接读写文件对象
<code>json.dumps(obj, sort_keys=True)</code>	键排序，便于人工对比文件

`ensure_ascii=True` 是默认值，中文会变成 `\uXXXX`
写配置文件时如果忘了 `ensure_ascii=False`，文件里会是一串 `\u5f85\u529e`。程序读起来没问题，但**人打开来看不懂**——而配置文件恰恰是给人改的。

读 JSON 文件时有个固定套路：**先判文件存在、再包 try**，缺文件不该是错误：

```
1 def load_app_state() -> dict:
2     if not STATE_PATH.exists():
3         return {}
4     try:
5         return json.loads(STATE_PATH.read_text(encoding="utf-8"))
6     except (OSError, ValueError) as exc:
7         logger.warning("状态文件读取失败（%s），按空白处理", exc)
8         return {}
```

▲ 由本手册编写；项目里 `login_memory.json` 的读取就是同一形状

11.4 sqlite3：本项目唯一的持久化方案

选择 SQLite 的理由很直接：**单文件、零配置、免安装**。对一个要给别人拷过去就能跑的桌面程序来说，这是唯一合理的选项。

连接与三个必设项

```
1 def __init__(self, db_path: Path):
2     self.db_path = db_path
3     self.conn = sqlite3.connect(str(db_path), check_same_thread=False)
4     self.conn.row_factory = sqlite3.Row
5     self.conn.execute("PRAGMA foreign_keys = ON")
```

▲ `todo_db.py` · 这三行是全项目数据层的统一开头

设置	为什么必须写
<code>row_factory = sqlite3.Row</code>	不说的话查询结果是普通元组，只能按 下标 取值。设了之后可以 <code>row["name"]</code> —— 加一列不会让所有取值错位
<code>PRAGMA foreign_keys = ON</code>	SQLite 默认不检查外键 （这是历史包袱），必须每个连接显式打开
<code>check_same_thread=False</code>	允许连接跨线程使用。本项目定时器与子线程都会读库

查询：参数化、避免 SQL 注入

```

1         rows = self.conn.execute(
2             "SELECT * FROM todo_holidays WHERE day LIKE ? ORDER BY day ASC",
3             (f"{year:04d}-{month:02d}-{day:02d}",),
4         ).fetchall()
5         return [Holiday.from_row(r) for r in rows]
```

▲ todo_db.py · 问号占位符 + 参数元组

永远不要用 f-string 拼 SQL

`conn.execute(f"... WHERE name = '{name}'")` 是**注入漏洞**，而且遇到名字里带单引号（比如「O'Brien」）会直接语法错误。

正确做法就是问号占位符：**SQL 语句是固定的，数据单独作为第二个参数传**。SQLite 驱动会自己做转义。命名占位符两种写法都支持：`?` 配元组、`:name` 配字典。

批量操作：executemany

```

1     def mark_alerted(self, moments: dict[int, str]):
2         """把「这一轮已经响过」记回条目（键为条目 id，值为提醒时刻原文）。"""
3         if not moments:
4             return
5         now = _now()
6         self.conn.executemany(
7             "UPDATE todo_items SET alerted_for = ?, updated_at = ? WHERE id = ?",
8             [(text, now, item_id) for item_id, text in moments.items()],
9         )
10        self.conn.commit()
```

▲ todo_db.py · executemany + 列表推导构造参数

`executemany` 接受「一个 SQL + 一批参数」，比循环调用 `execute` 快得多（少了每轮的语句准备开销）。注意这里只 **commit 一次**，而不是每条 `commit` —— 后者会让 SQLite 做 N 次磁盘刷写。

建表：IF NOT EXISTS + executescript

```

1     def create_tables(self):
2         """建表（IF NOT EXISTS，重复调用安全）。"""
3         self.conn.executescript("""
4             /* 清单：颜色 + 图标，是「用不同颜色区分」的载体 */
5             CREATE TABLE IF NOT EXISTS todo_lists (
6                 id            INTEGER PRIMARY KEY AUTOINCREMENT,
7                 name          TEXT    NOT NULL,
8                 color          TEXT    NOT NULL DEFAULT '#007AFF',
9                 icon           TEXT    NOT NULL DEFAULT 'list',
10                sort_order     INTEGER NOT NULL DEFAULT 0,
11                created_at      TEXT    NOT NULL,
12                updated_at      TEXT    NOT NULL
13            );
14
15            CREATE INDEX IF NOT EXISTS idx_todo_items_list
16                ON todo_items (list_id, completed, due_date);
17        """)
18        self.conn.commit()

```

▲ todo_db.py · 略作节选

IF NOT EXISTS 让建表变成幂等操作

`__init__` 里直接调 `create_tables()`，不需要先判断「数据库是新的还是老的」。这同样是幂等思想：把「首次运行」和「再次运行」写成同一条路径，就不会出现「第二次启动报错」这类只在特定顺序下出现的问题。

数据库迁移：SQLite 没有 ADD COLUMN IF NOT EXISTS

```

1     existing = {
2         row["name"]
3         for row in self.conn.execute("PRAGMA table_info(todo_items)").fetchall()
4     }
5     for column, ddl in (
6         ("alerted_for", "ALTER TABLE todo_items ADD COLUMN "
7                        "alerted_for TEXT NOT NULL DEFAULT ''"),
8         ("snooze_until", "ALTER TABLE todo_items ADD COLUMN "
9                          "snooze_until TEXT NOT NULL DEFAULT ''"),
10    ):
11         if column not in existing:
12             self.conn.execute(ddl)
13     self.conn.commit()

```

▲ todo_db.py · 加字段的标准姿势：先查后补

迁移的两条纪律

- ① **新字段必须有默认值**（`NOT NULL DEFAULT ''`）—— 否则老数据那些行就没有合法值。
- ② **加列永远只做加法**：不改名、不删列、不改类型。SQLite 的 `ALTER TABLE` 能力很弱（只支持 `ADD COLUMN` 和 `RENAME`），要改结构就得「建新表 + 拷数据 + 换名」，那是另一个量级的风险。所以设计表时多留一个空列的成本，远低于事后改结构。

排序：多级 ORDER BY

```
1 "SELECT * FROM todo_lists ORDER BY sort_order ASC, id ASC"
```

▲ todo_db.py · fetch_lists()

第二级 `id ASC` 是**稳定排序**的保证：`sort_order` 相同的两条按 `id` 排，结果不会随数据库内部顺序变化。没有它，用户会看到「顺序偶尔自己变」。这与 4.6 节里 Python 侧「用下标做第二排序键」是同一个道理。

事务与 commit

对象	作用
<code>conn.commit()</code>	提交当前事务，落盘
<code>conn.rollback()</code>	回滚未提交的改动
<code>with conn:</code>	进入时开事务，正常退出自动 <code>commit</code> 、异常自动 <code>rollback</code>
<code>conn.execute</code> 直接改数据	SQLite 驱动会 隐式开启 事务，等你 <code>commit</code>
<code>conn.close()</code>	关闭连接；未提交的改动会丢

本项目手工 commit 而不依赖 with conn:

因为待办模块的连接是跨页面共享的长连接，用 `with conn:` 会频繁开关事务，而界面操作往往是「读一下、改一下、再读一下」的连续动作，中间开事务反而容易撞上锁。

项目的做法是：**每个写方法自己以 `self.conn.commit()` 收尾**，读方法完全不碰事务。这样「一次界面操作 = 一次 `commit`」，语义清楚。

取行数据的几种方式

写法	结果	取不到时
<code>fetchall()</code>	全部行组成的列表	空列表
<code>fetchone()</code>	第一行	None
<code>fetchmany(n)</code>	最多 <code>n</code> 行	可能是短列表
<code>row["列名"]</code>	按列名取值（需设 <code>row_factory</code> ）	<code>KeyError</code>
<code>row.keys()</code>	列名列表	—
<code>conn.total_changes</code>	本连接累计改动行数	—
<code>cursor.lastrowid</code>	刚插入行的 <code>id</code>	—

```

1         cur = self.conn.execute(
2             "INSERT INTO todo_items (list_id, title, created_at, updated_at) "
3             "VALUES (?, ?, ?, ?)",
4             (list_id, title, now, now),
5         )
6         self.conn.commit()
7         return int(cur.lastrowid)

```

▲ 插入之后拿回新 id 的写法

11.5 logging：一份日志基建的最小正确实现

```

1 def setup_logging(log_dir=None, level=None, *, console=None,
2                  filename=LOG_FILENAME):
3     """配置 root logger，重复调用幂等。返回 root logger。"""
4     global _configured
5     root = logging.getLogger()
6     if _configured:
7         return root
8
9     resolved = _resolve_level(level)
10    formatter = logging.Formatter(_FORMAT, datefmt=_DATEFMT)
11    handlers = []
12
13    if log_dir is not None:
14        try:
15            log_dir = Path(log_dir)
16            log_dir.mkdir(parents=True, exist_ok=True)
17            file_handler = logging.handlers.RotatingFileHandler(
18                log_dir / filename,
19                maxBytes=MAX_BYTES,
20                backupCount=BACKUP_COUNT,
21                encoding="utf-8",
22            )
23            file_handler.setFormatter(formatter)
24            file_handler.setLevel(resolved)
25            handlers.append(file_handler)
26        except OSError as exc:
27            # 只读介质 / 权限不足等：继续走控制台，不中断启动
28            try:
29                sys.stderr.write(f"[log_setup] 日志文件不可用（{exc}），仅输出到控制台\n")
30            except Exception:
31                pass

```

▲ log_setup.py · 节选

要点	做法
配 root logger	入口调一次，之后任意模块 <code>getLogger(__name__)</code> 自动生效
幂等	<code>_configured</code> 标志，重复调用直接返回，不会重复挂 handler
必须挂文件 handler	打包版 <code>console=False</code> ， <code>sys.stderr</code> 是 <code>None</code> ，只挂控制台等于日志全丢

要点	做法
轮转	<code>RotatingFileHandler(maxBytes=1MB, backupCount=3)</code> ，不会把磁盘写满
编码	<code>encoding="utf-8"</code> ，中文日志在 Windows 上才不会乱码
失败只降级	日志基建不该成为程序起不来的原因

控制台 handler 要「先探测再挂」

打包成窗口程序后 `sys.stderr` 是 **None**。此时若照旧挂 `StreamHandler(sys.stderr)`，Python 的 logging 每次 `emit` 都会抛 `AttributeError: 'NoneType' object has no attribute 'write'` ——然后被 logging 自己吞掉。表面平静，实际一条日志都没落。

项目的做法是写一个 `_console_stream()`：探测 `sys.stderr is None` 就返回 `None`，顺便试写一次确认可用，再去挂 handler。

```

1  logger = logging.getLogger(__name__)      # 每个模块顶部一行，不需要任何配置
2
3  # 记录异常：自动带上堆栈
4  try:
5      risky()
6  except Exception:
7      logger.exception("待办到点提醒巡检失败")
8
9  # 记录普通信息：用 %s 占位符，不要提前拼字符串
10 logger.debug("日志已初始化：级别=%s，处理器=%d 个", level, len(handlers))

```

▲ 模块内的用法

`logger.info("..." + str(x))` 与 `logger.info("...%s", x)` 的区别

后者只有真的要输出时才会做字符串格式化。日志级别是 INFO 时，一条 DEBUG 级别的格式化完全不会执行——在高频路径上这是实打实的性能差别。

项目里因此全用 `%s` 占位符形式，没有一处提前拼字符串。

`logger.exception` 只能用在自己没被处理过的异常分支里

它内部调 `sys.exc_info()`，必须处在 `except` 块中才有堆栈可打。在 `except` 块外调用它，只是记一条 ERROR 而没有堆栈——看起来一样，但排查时才发现少了最关键的信息。

11.6 re：正则的四个实用要点

```

1  _INLINE_CODE = re.compile(r"~([~]+)~")
2  _BOLD = re.compile(r"\*~*([~*]+)~*~")
3
4  for m in PATTERN.finditer(text):
5      spans.append((m.start(), m.end(), m.group(1)))

```

▲ 本手册生成器里的用法；项目里 `markdown_view.py` 同形

要点	说明
先编译	<code>re.compile()</code> 一次，循环里复用； <code>re</code> 内部虽有缓存，显式更明确
用 <code>finditer</code> 而不是 <code>findall</code>	<code>finditer</code> 给的是 <code>Match</code> 对象，能拿到 <code>start()</code> / <code>end()</code> / 分组
用 <code>re.escape()</code> 转义字面量	搜用户输入或文件名时必做，否则 <code>*</code> 会被当语法
原始字符串	正则一律写 <code>r"..."</code> ，否则反斜杠要先过 Python 一层

多分组正则 + `findall` 会得到元组列表

`re.findall(r"(\r\n|\n)", text)` 返回的是**元组列表**而不是字符串列表，于是 `hits[0]` 拿到的是 `(' \n',)`。本项目在写字节补丁脚本时踩过一次。

要精确控制就一律用 `finditer`： `for m in re.finditer(pat, text)`，然后 `m.group(0)` 取整体、`m.group(1)` 取第一个分组，还能用 `m.start()` / `m.end()` 定位。

另一个与字节打交道时的常见错误：**在字节串上做正则时，模式也要是字节串**。项目里查文件里有没有三引号的脚本就是把模式和 `re.escape()` 都编码成 `bytes`：

```
1 for m in re.finditer(re.escape(b'\"\"\"'), raw):
2     ...
```

▲ 在 `bytes` 上做正则，模式和输入都必须是 `bytes`

11.7 其它常用模块

模块	用途	项目里的例子
<code>os</code>	环境变量、路径	<code>os.environ.get(ENV_LEVEL, "")</code> 覆盖日志级别
<code>sys</code>	解释器相关	<code>sys.frozen</code> 判打包、 <code>sys.path</code> 加搜索路径
<code>subprocess</code>	启动外部程序	工具包启动 <code>exe</code> 、 <code>explorer</code> 打开目录
<code>shutil</code>	文件复制删除	导入图片时复制到数据目录
<code>webbrowser</code>	打开浏览器	点链接打开文档
<code>zipfile</code>	压缩包	读 Excel (xlsx 本质是 zip)
<code>hashlib</code>	摘要	密码哈希 <code>hashlib.sha256(...).hexdigest()</code>
<code>calendar</code>	日历计算	<code>calendar.monthrange()</code> 取月末
<code>time</code>	计时	<code>time.perf_counter()</code> 测耗时
<code>ctypes</code>	调 Windows API	单实例互斥、 <code>PrintWindow</code> 截图

```
1 def hash_password(password: str) -> str:
2     return hashlib.sha256(password.encode("utf-8")).hexdigest()
```

▲ `main.py` · 密码哈希（自用工具的强度取舍，见下方说明）

SHA-256 直接哈希密码，在生产环境是不够的

没有加盐、没有慢哈希，字典攻击很容易。本项目是单机自用工具、只防「别人翻开数据库文件能看到明文」，这个强度按需求是够的。

真要正经存密码，用 `hashlib.scrypt` 或 `bcrypt`，它们把每一条密码的哈希成本抬到几十毫秒，暴力破解就不可行了。这一段的价值不在代码，而在「知道自己选的是什么强度」。

```
1 result = subprocess.run(  
2     [str(exe_path), "--version"],  
3     capture_output=True, text=True, timeout=15,  
4     creationflags=subprocess.CREATE_NO_WINDOW,    # 不弹黑框  
5 )  
6 if result.returncode == 0:  
7     version = result.stdout.strip()
```

▲ 运行外部命令并取输出（参数用列表形式，不要拼成一个字符串）

`subprocess` 的三条纪律

- ① 参数用列表：`["powershell.exe", "-Command", cmd]`。用 `shell=True` 加拼字符串等于把命令行注入的口子开着。
- ② 一定要 `timeout`：外部程序卡住会连带界面卡死。
- ③ Windows 上给 `CREATE_NO_WINDOW`：否则每跑一次都会闪一个黑框，对一个 GUI 程序来说非常难看。

第十二章 GUI 相关的 Python 语法

这一章不讲界面设计（那是另一个话题），只讲用 Python 写 Tkinter 时会用到的语言特性：回调、闭包、定时器、事件、变量类，以及控件生命周期带来的那几个专属异常。

12.1 界面为什么用类组织

```
1 class TodoPage(ttk.Frame):
2     def __init__(self, master, db=None, *, on_status=None):
3         super().__init__(master)
4         self.db = db
5         self.current_list_id = 0
6         self._list_rows = {}
7
8         self._build()
9         self.refresh_all()
```

▲ todo_page.py · 页面类的构造骨架

用类的理由有两条，都很实际：

1. **状态要有地方放。** `self.current_list_id`（当前选中的清单）、`self._list_rows`（行控件索引）必须活到下一次交互——装在函数局部变量里，函数一返回就没了。
2. **回调要能找回来。** 按钮的 `command=self.on_click` 是一个绑定方法，它自带 `self`，所以点下去时能拿到这套状态。用纯函数就得靠全局变量。

为什么页面继承 `ttk.Frame` 而不是 `object`

`ttk.Frame` 本身就是一个容器控件。继承它之后，`TodoPage` 实例可以直接 `pack()` / `grid()` 到父容器里，就和一个普通 `Frame` 一样用：`self.todo_view = TodoPage(container, db); self.todo_view.pack(fill="both")`。如果继承 `object`，就得自己再持有一个 `Frame` 并把它暴露出去，多一层转发。

注意构造签名里的 `db=None`：数据层是「注入」进来的，不是页面自己 `new` 的。这是 7.8 节「组合」的直接收益——测试时塞一个替身进去就行，不需要真数据库。

12.2 回调：传函数对象，不传调用结果

```
1 dialog = TodoAlertDialog(
2     self, items, db=self.todo_db,
3     on_changed=self._after_todo_alert,    # 传函数对象，不加括号
4     on_open=self._open_todo_item,
5 )
```

▲ main.py · 三个回调全是具名方法名

正确	错误	错误的结果
<code>command=self.save</code>	<code>command=self.save()</code>	绑定时就执行一次，回调变成 <code>None</code>
<code>bind("<Enter>", self._on_enter)</code>	<code>bind("<Enter>", self._on_enter())</code>	同上
<code>command=lambda: self.save(1)</code>	<code>command=lambda x: self.save(1)</code>	参数对不上，点击时报错
<code>bind("<Enter>", lambda e: f(e))</code>	<code>bind("<Enter>", lambda: f(e))</code>	Tkinter 会传事件参数，参数个数不符

bind 的回调一定会收到一个事件对象

`widget.bind("<Enter>", handler)` 里的 `handler` 会被 Tkinter 调用成 `handler(event)`。所以回调签名必须能收下 一个参数 —— 这就是为什么项目里到处是 `lambda e: self.set_hover(True)` 里的那个 `e`。

如果用的是具名方法（`self._invoke`），它的签名就得是 `def _invoke(self, event=None)`。写成 `def _invoke(self)` 也能跑 `command=`，但接 `bind` 就会 **`TypeError`** —— 所以项目里统一给事件处理方法加 `event=None`。

```
1 self.bind("<Enter>", lambda e: self.set_hover(True))
2 self.bind("<Leave>", lambda e: self.set_hover(False))
3 if command is not None:
4     self.bind("<Button-1>", self._invoke)
```

▲ ui_components.py · 两种回调用法并存

12.3 after: Tkinter 的定时器

```
1 TODO_TICK_MS = 30 * 1000          # 30 秒
2
3 def todo_alert_check(self):
4     """巡检一次「有没有提醒该响了」。
```

▲ main.py · 自续期定时器



▲ 就是上面那个 30 秒定时器弹出来的东西：屏幕右下角的到点提醒窗。`-topmost` 压过别的窗口，但不 `grab_set`，所以不会把正在打字的人从输入框里拽出来。

方法	作用
<code>widget.after(ms, func)</code>	延迟 ms 毫秒调用一次
<code>widget.after(ms, func, *args)</code>	带参数调用
<code>widget.after_idle(func)</code>	界面空闲时调用
<code>handle = widget.after(...)</code>	返回句柄，可以取消
<code>widget.after_cancel(handle)</code>	取消（退出前必须做，否则回调会打到已销毁的控件上）
<code>widget.after_cancel("all")</code>	取消该控件上全部定时任务
<code>widget.update() / update_idletasks()</code>	强制立刻处理一次事件队列（截图、测试时用）

自续期定时器的三条纪律

- ① 续期必须放 `finally` —— 否则一次异常就让定时器永久死掉（见 8.1）。
- ② 退出前要停 —— 项目里用 `self._exiting` 标志：退出流程一开始就置位，下一轮巡检看到标志直接 `return`，不会再去碰正在销毁的控件。
- ③ 回调出错要有人接住 —— Tkinter 的事件循环里抛出的异常不会让程序退出，只会打到 `stderr`（打包版等于吞掉）。所以回调里必须自己 `try/except` + 记日志。

为什么不用 `threading.Timer` 或独立线程

Tkinter 不是线程安全的：只有创建控件的那一个线程能碰控件。从子线程里调 `widget.config(...)` 可能看起来正常、也可能在几分钟后随机崩掉 —— 这类 bug 极难复现。
`after()` 跑在主线程的事件循环里，天然没有这个问题。**要在界面程序里做定时任务，就用 `after`。**

12.4 事件绑定与事件对象

事件	触发
<Button-1> / <ButtonRelease-1>	鼠标左键按下 / 松开
<B1-Motion>	按住左键拖动
<Double-Button-1>	双击
<Enter> / <Leave>	鼠标进入 / 离开控件
<KeyRelease>	按下一个键
<Return>	回车
<FocusOut>	失去焦点（表单自动保存常绑它）
<Configure>	控件尺寸变化
<<ComboboxSelected>>	虚拟事件（ttk 特有），注意是 双尖括号

```
1 class _DragEvent:
2     """拖动回调用得到的字段就这两个；自己造一个替身，测试时就不需要真鼠标。"""
3
4     def __init__(self, y_root: int):
5         self.y_root = y_root
6
7
8 page._list_drag_press(ids[0], _DragEvent(base + mid(0)))
9 page._list_drag_motion(ids[0], _DragEvent(base + mid(2)))
10 page._list_drag_release(ids[0], _DragEvent(base + mid(2)))
```

▲ scripts/test_todo_ui.py · 用替身对象模拟事件

这段是**测试写法**上的一个要点：拖动处理只用到事件对象上的 `y_root` 一个字段，那么测试就不必真的造鼠标事件，传一个只有该字段的小对象即可。这叫「鸭子类型」的实用面——**不要求参数是某个类，只要求它有那个属性**。结果是拖动的回归测试完全不需要真实鼠标操作，跑起来稳定又快。

事件对象上常用的字段

`event.x / event.y` 是**相对控件左上角**的坐标；`event.x_root / event.y_root` 是**相对屏幕**的坐标；`event.widget` 是触发事件的控件；`event.keysym` 是按键名（如 `"Return"`）；`event.width / event.height` 在 `<Configure>` 里给出新尺寸。

拖动计算要用 `y_root`（屏幕坐标），因为它不受「被拖动的行自己在移动」影响；用 `event.y` 会在行移动之后跟着漂。

12.5 变量类：StringVar / IntVar / BooleanVar

```
1         self.keyword_var = tk.StringVar()
2         entry = ttk.Entry(bar, textvariable=self.keyword_var)
3         entry.bind("<KeyRelease>", lambda e: self._on_search())
4         ...
5     def _on_search(self):
6         keyword = self.keyword_var.get().strip()
```

▲ todo_page.py · 搜索框的两种用法并存

写法	取值	适用
<code>textvariable=var</code>	<code>var.get()</code> / <code>var.set(x)</code>	需要程序主动改控件内容时
直接 <code>entry.get()</code>	从控件读	只读用户输入时更简单
字符串	<code>tk.StringVar()</code>	文本
整数	<code>tk.IntVar()</code>	复选框、计数器
布尔	<code>tk.BooleanVar()</code>	<code>Checkbutton</code>
双击	<code>tk.DoubleVar()</code>	滑块数值

变量类是「双向绑定」的实现方式

`textvariable=` 之后，改 `var` 会立刻改控件，改控件也会立刻改 `var`。本项目只在「需要程序回写控件」的地方用它（比如搜索框要能被清空）。纯读取输入的场景直接 `entry.get()` 更省事 —— 少一个需要维护的对象。一个易错点：`IntVar` 在输入框为空或非数字时 `get()` 会抛 `TclError`，因为 Tk 存的是字符串、转 `int` 失败。所以读数字一律要包一层兜底。

12.6 控件生命周期与 `TclError`

```
1     key = tuple(base)
2     cached = _STRIKE_FONTS.get(key)
3     if cached is not None:
4         try:
5             cached.actual()           # 探一次活：解释器换过之后旧对象就是废的
6             return cached
7         except tk.TclError:
8             _STRIKE_FONTS.pop(key, None)
```

▲ todo_page.py · `strike_font()` 的缓存探活

控件被销毁之后，任何 `cget / config` 都会抛 `TclError`

报错长得像 `TclError: invalid command name ".!todopage.!frame.!label3"`。触发场景很多：**定时器回调打在已关闭的窗口上、在 Tk 解释器销毁后复用缓存的 `Font` 对象、跨测试用例复用了上一个 `Tk()` 根窗口的控件。**

应对方式有两种，项目里都用了：

- ① **先判存在**：`if widget.winfo_exists(): widget.destroy()`；
 - ② **探活后再用**：像上面这样先调一个无害方法，抛 `TclError` 就说明对象已废，清掉缓存重建。
- 永远不要缓存 Tk 对象而不做存活检查** —— 本项目的字体缓存就是这么修的。

```
1 def _close_todo_alert(self):
2     """关掉还挂着的提醒窗（开会错过的那一批不应一直压在最上层）。"""
3     dialog = getattr(self, "_todo_alert_win", None)
4     self._todo_alert_win = None          # 先置空：让本方法可重复调用
5     if dialog is None:
6         return
7     try:
8         if dialog.winfo_exists():
9             dialog.destroy()
10    except Exception:
11        pass
```

▲ `main.py` · 关闭窗口的标准姿势：先置空引用、再判存在、最后包 `try`

winfo 系列：询问控件的实际状态

方法	返回
<code>winfo_exists()</code>	控件是否还活着
<code>winfo_width() / winfo_height()</code>	当前实际尺寸（ 布局算完之后才是真实值 ）
<code>winfo_reqwidth() / winfo_reqheight()</code>	控件自己请求的尺寸
<code>winfo_x() / winfo_y()</code>	相对父容器的 左上角坐标
<code>winfo_rootx() / winfo_rooty()</code>	相对屏幕的坐标
<code>winfo_children()</code>	子控件列表
<code>winfo_screenwidth() / winfo_screenheight()</code>	屏幕尺寸
<code>winfo_ismapped()</code>	是否真的被显示出来了

`winfo_width()` 在布局计算前返回的是 1（不是错，是还没算）

刚 `pack()` 完立刻问尺寸，拿到的往往是 1。必须等 Tk 处理过一次事件队列，也就是调用 `update_idletasks()` 或 `update()` 之后才有真实值。

测试里所有「量尺寸」的断言前面都必须先 `update()`，否则断言的是「还没布局」而不是「布局结果」。

另外 `winfo_ismapped()` 有个特别之处：**主窗口 `withdraw()` 时，所有控件都返回 0**。所以拿它判断「控件有没有被饿死」之前，必须先 `deiconify()`。

12.7 一个跨语言的坑：Tk 会「饿死」控件

```
1         self.vsb = ttk.Scrollbar(self, orient="vertical")
2         self.vsb.pack(side="right", fill="y")           # 先 pack 滚动条
3
4         self.canvas = tk.Canvas(self, bg=bg, highlightthickness=0, bd=0,
5                                   yscrollcommand=self.vsb.set)
6         self.canvas.pack(side="left", fill="both", expand=True) # 后 pack 画布
```

▲ todo_page.py · ScrollArea 里 pack 的顺序

这个坑不属于 Python 语法，但每次都让人以为是语法问题

`pack` 是按调用顺序分配剩余空间的。如果先 `pack` 一个 `fill="both", expand=True` 的控件，它会把空间全部吃掉，后面 `pack` 的固定尺寸控件只分到 **0 像素**，于是 Tk **根本不映射它**——代码明明写了，界面上就是不存在。这不是「被裁掉一点」，而是**完全不存在**。判定口径是：`wininfo_manager()` 非空 且 `wininfo_ismapped() == 0`。
规矩：先 pack 固定尺寸的控件，最后 pack 那个 `expand=True` 的内容区；滚动条要先于它管理的滚动区域。

本章小结：GUI 里最该记住的四条

- ① 回调传函数对象（`self.save`），不传调用结果（`self.save()`）；
- ② 定时任务用 `after + finally` 续期 + 退出标志，不要用线程；
- ③ 任何缓存的 Tk 对象在用之前都要探活，控件销毁后一律 `TclError`；
- ④ 别在模块顶层建窗口或控件——那会让 `import` 有副作用，测试也没法隔离。

第十三章 工程约定与陷阱

这一章是那些**不属于任何单一语法点、但每个真实项目都会撞上**的东西。它们大多与「Python 在 Windows 上 + 中文 + 打包发布」这个组合有关。

13.1 行尾：CRLF 与 LF

场景	行尾	说明
Windows 上的多数编辑器	CRLF (<code>\r\n</code>)	本项目应用源码就是这种
Git Bash / 跨平台工具	LF (<code>\n</code>)	本项目 <code>scripts/*.py</code> 与 <code>*.md</code> 是这种
Python 文本模式读取	统一翻译成 <code>\n</code>	<code>universal newlines</code> ，看不见 CRLF
Python 文本模式写入	<code>\n</code> 被翻译成 平台默认	Windows 上就是 CRLF —— 这是 <code>write_text</code> 的陷阱
Python 二进制模式	原样进出	字节级补丁必须走这条

关键点是第二行与第四行：**Python 的文本模式会让行尾「看起来统一」**，所以你在代码里从没感觉到 CRLF 的存在；但**写回磁盘时它又会按平台转换**，于是「读出来再写回去」这个动作本身就会改文件。

`Path.write_text` 在 Windows 上会把整个文件变成 CRLF

本项目实际踩过：一个纯 LF 的测试脚本用 `write_text` 写回之后，946 行全部变成 CRLF，`git diff` 显示「946 增 / 778 删」，真实的逻辑改动被淹没在里面。

要保住原行尾只有一条路：`read_bytes()` + `write_bytes()`，中间需要处理文本时用 `decode("utf-8")`，处理完再 `encode("utf-8")`，全程不让行尾经过文本模式的翻译。

```
1 raw = p.read_bytes() # 字节读：CRLF 保持原样
2 assert raw.count(b"\r\n") == 0, "这个文件应该是纯 LF" # 先确认现状
3 before = raw.count(b"\n")
4 text = raw.decode("utf-8") # 现在是字符串，里面的换行是 "\n"
5 text = text.replace(old, new, 1) # 只改目标片段
6 out = text.encode("utf-8") # 字节写回：不产生任何 CRLF
7 assert out.count(b"\n") == before + added # 行数变化符合预期
8 p.write_bytes(out)
```

▲ 纯 LF 文件的补丁骨架

完整规则（本项目所有改写源码的操作都遵守）

- 1. 先查明行尾现状：raw.count(b"\r\n") 与 raw.count(b"\n") 相等 → 纯 CRLF；后者减前者得到的就是「单独 LF」的数量。
- 2. 纯 CRLF 文件：text.replace("\n", "\r\n") 写回，并断言 out.count(b"\r\n") == out.count(b"\n")。
- 3. 纯 LF 文件：绝对不要经过 write_text，只走 write_bytes。
- 4. 混排文件（如 main.py）：先记下「单独 LF」的数量（167 个），补丁锚点带上 \r\n，写完复核这个数量没变。
- 5. 收尾一律用 git diff --numstat 复核变更行数 —— 数字不对说明行尾被动过。

13.2 文本 vs 字节：什么时候必须用字节

任务	该用的 API	理由
读写配置 / 日志	read_text / write_text	人看的文本，行尾无所谓
改写源码文件	read_bytes / write_bytes	必须保住原行尾
解析二进制格式	read_bytes	图片、Excel、zip
网络 / 子进程 IO	bytes	协议是字节流
哈希、加解密	bytes	算法定义在字节上
写 CSV / 文本导出	open(..., newline="")	让 csv 模块自己管行尾

csv 模块必须配 newline=""

csv.writer 自己会写 \r\n，而文本模式又会把 \n 再翻一遍，结果是每行多一个空行。正确写法是 open(path, "w", encoding="utf-8-sig", newline="")。这是 Python 官方文档特别标注的一条。

encoding="utf-8-sig" 是什么时候该用

utf-8-sig 会在文件开头写 BOM (EF BB BF)。Excel 打开 CSV 时

靠这个 BOM 判断编码，没有它中文会乱码。所以：给用户用 Excel 打开的 CSV 用 utf-8-sig；程序自己读写的 JSON / 日志 / 源码统统用普通 utf-8（加 BOM 反而会让严格的解析器报错）。

本项目 tools_page.py 里的 CSV 导出就是这么分的。

13.3 编码：统一 UTF-8，一处都不要例外

```
1 # 源码文件
2 # -*- coding: utf-8 -*-
3
4 # 读写文件
5 path.read_text(encoding="utf-8")
6 path.write_bytes(text.encode("utf-8"))
7
8 # 日志
9 RotatingFileHandler(..., encoding="utf-8")
10
11 # 子进程
12 subprocess.run(..., encoding="utf-8", errors="replace")
13
14 # 控制台（打包版 stderr 可能没有）
15 stream.reconfigure(errors="replace")
```

▲ 项目里的编码设置集中在这几处

坑	表现	应对
Windows 默认编码是 cp936	读写文件不指定 encoding 时中文可能乱码或报错	每个 open 都写 encoding="utf-8"
控制台打印中文	UnicodeEncodeError 或显示问号	errors="replace" 放宽
BOM	JSON 解析失败、字符串开头多一个不可见字符	程序内部一律普通 utf-8
源码里有非法字节	SyntaxError: Non-UTF-8 code	确保编辑器保存为 UTF-8 无 BOM

13.4 时间：本项目明确选了「本机时间」

做法	含义	适用
<code>datetime.now()</code>	本机当前时间，naive	单机自用工具（本项目）
<code>datetime.utcnow()</code>	UTC 时间，naive	已不推荐，用下面那个
<code>datetime.now(timezone.utc)</code>	带时区的 UTC	需要跨机器一致时
<code>datetime.now().timestamp()</code>	Unix 时间戳（秒）	存库、跨系统比较
<code>date.today()</code>	本机日期	只关心「哪一天」时

naive 时间的边界在哪

naive datetime 的含义是「这台机器认为的本地时间」。它有三种情况会出问题：

- ① **换机器**：同一份数据在另一个时区的电脑上打开，时间含义就变了；
- ② **夏令时**：本地时间在切换日会出现重复或缺失的一小时；
- ③ **与外部系统对时间**：服务器给的是 UTC，直接比较会差几小时。

本项目的场景是「单机、自用、只在本地比较」，所以 naive 时间成立。**但这个结论要写下来**（就像这条注释一样），否则将来接了云同步功能，没人知道当初是「有意选的」还是「没想到」。

13.5 数值：先把误差摆到台面上

```
1 print(0.1 + 0.2)           # 0.30000000000000004
2 print(0.1 + 0.2 == 0.3)    # False
3 print(round(2.675, 2))     # 2.67 —— 不是四舍五入的错，是 2.675 本身就存不准
4 print(1 / 3)               # 0.3333333333333333
```

▲ 这些结果都是「正确」的，浮点数就是这样

需求	做法
显示两位小数	<code>f"{v:.2f}"</code>
比较两个浮点是否相等	<code>abs(a - b) < 1e-9</code>
金额计算	用 <code>int</code> 存分，或 <code>decimal.Decimal("0.1")</code>
只要整数结果	<code>//</code> （向下取整）、 <code>int()</code> （向零取整）、 <code>round()</code> （银行家舍入）
统计求和顺序敏感	<code>math.fsum()</code> 精度更好

`round()` 是「银行家舍入」，不是四舍五入
`round(0.5)` 得 0、`round(1.5)` 得 2 —— 它舍入到最近的偶数。这是 IEEE-754 的默认策略，用来避免大量数据相加时的系统性偏差。要真正的四舍五入，用 `decimal.Decimal(...).quantize(Decimal("0.01"), rounding=ROUND_HALF_UP)`。

13.6 静态检查：pyflakes 与它的口径

工具	查什么	本项目
pyflakes	未使用的导入 / 变量、未定义的名字、f-string 问题	在用，作为回归基线
ruff	pyflakes + pycodestyle + 很多规则，极快	待办清单上
mypy / pyright	类型检查	待办清单上
black	格式化	未用（现有风格已统一）

pyflakes 的口径要限定在「有效范围」
项目根目录有一批 `_` 开头的临时脚本，噪声很大，把它们算进来会让告警数虚高。本项目的有效基线只统计「非 `_` 开头 + `embedded_admin_tools/`」，当前是 7 条旧账，全是「赋值未使用」这类无关痛痒的。
关键不是「零告警」，而是「告警数不再增加」—— 回归时对比数字，新增一条就要查。

本项目补的两个专用检查脚本

`pyflakes` 有两个盲区，项目为此写了单独脚本：

`check_name_scope.py` —— 查 `except` 块里的名字在块外被引用（Python 3 会在 `except` 结束时删除 `except E as e` 里的 `e`）以及闭包作用域问题；

`project_audit.py` —— 用 AST 统计函数长度、嵌套深度、重复代码。

`pyflakes` 不是全覆盖，知道它漏什么，比用到它更重要。

13.7 打包：Python 特有的三个认知

1	ExpiryManager_fixed.spec	
2	├── Analysis(['main.py'])	分析依赖图，收集所有 import
3	├── PYZ	把 .pyc 压进一个归档
4	├── EXE	bootloader + PYZ + 数据文件
5	└── (onefile 模式没有 COLLECT 目录，全部塞进单个 exe)	

▲ PyInstaller onefile 的产物结构

认知	后果
漏了模块不报错	只在运行版炸。所以要「反向核对」：打开 exe 归档验证符号在不在
<code>hiddenimports</code> 要手工补	动态导入（ <code>importlib</code> 、字符串名字）分析不到，必须列出
路径会变	<code>__file__</code> 指向临时解包目录，数据目录要用 <code>sys.executable</code> 定位
没有控制台	<code>console=False</code> 时 <code>sys.stderr</code> 是 <code>None</code> ，日志必须落文件
首次启动慢	onefile 要先解包到临时目录，体积越大越慢

「反向核对」怎么做

打开 exe 的归档，把源码编译后的字节码取出来，检查关键函数名在不在：

`CArchiveReader(exe) → open_embedded_archive("PYZ.pyz") → extract("todo_db")`，然后递归遍历 `code object` 的 `co_names` / `co_consts`（`repr(code)` 看不到函数名）。入口脚本 `main` 不在 `PYZ` 里，得从 `CArchive` 的 `main` 条目取。

本项目把它固化成 `scripts/check_exe.py`，每次打包后跑一遍。

打包脚本最大的危险是「删掉真实数据」

很多打包脚本第一步是 `rmdir /s /q dist`，之后只把少数几个文件拷回去——于是用户积累的运行期数据永久丢失（数据库、上传的图片、登录记忆）。本项目因此不用那个脚本，改用「只备份数据库与工具目录到临时位置、只删 exe」的安全版本。

任何「清理后重建」的脚本，都要先问一句：重建得回来吗？

附录 A · 语法速查卡

这份附录是一页纸级别的浓缩。适合打印出来放在手边，或者在你「明明记得有这么一个语法但想不起来怎么写」的时候翻。

A.1 数据类型与字面量

	字面量	可变	可哈希	常用取值
int	42、0x1F、0b1010、1_000_000	否	是	整数
float	3.14、1e-9	否	是	小数
bool	True / False	否	是	int 的子类
str	'a'、"a"、f"{x}"、r"\d"	否	是	文本
bytes	b"abc"	否	是	字节日志、二进制
list	[1, 2]	是	否	有序收集
tuple	(1, 2)、1, 2	否	是	固定组合、字典键
dict	{'a': 1}	是	否	键值映射（保插入序）
set	{1, 2}、set()	是	否	去重、存在性
None	None	否	是	「没有值」
frozenset	frozenset({1})	否	是	不可变集合

A.2 字符串与容器操作

类别	操作
字符串	s.strip() s.split(sep) s.replace(a, b) s.startswith(p) s.endswith(p) s.casefold() s.zfill(2) s.join(seq) s in t s[1:5] s[::-1]
列表	append extend insert remove pop clear index count sort reverse copy
字典	d.get(k, dflt) d.setdefault(k, v) d.update(o) d.pop(k, dflt) d.keys() d.values() d.items() d1 d2
集合	add discard remove union intersection difference issubset
通用	len(x) x in c max/min(c) sum(c) sorted(c) reversed(c) enumerate(c) zip(a, b) any(c) all(c) list(c) tuple(c) set(c) dict(pairs)

A.3 控制流与函数

写法	
条件	<code>if / elif / else</code> ; 三目 <code>a if c else b</code>
循环	<code>for x in c:</code> / <code>while c:</code> ; <code>break / continue / pass</code> ; <code>for ... else:</code>
异常	<code>try / except E / else / finally</code> ; <code>raise E(...)</code> / <code>raise ... from e</code>
函数定义	<code>def f(a, b=1, *args, c, **kw) -> T:</code>
仅关键字	<code>def f(a, *, b):</code> —— <code>b</code> 只能用 <code>b=</code> 传
返回多值	<code>return a, b</code> (实为元组)
lambda	<code>lambda x: x + 1</code> —— 只能是一个表达式
装饰器	<code>@deco</code> 写在 <code>def</code> 上方; <code>functools.wraps</code> 保住元信息
闭包	内层函数读外层变量; 写要 <code>nonlocal / global</code> ; 绑定用 <code>lambda i=i:</code>

A.4 类

写法	
定义	<code>class C(Base):</code>
构造	<code>def __init__(self, ...):</code> 第一句是 <code>super().__init__(...)</code>
实例 / 类属性	<code>self.x = 1</code> / 类体里 <code>x = 1</code>
计算属性	<code>@property def x(self):</code> 用 <code>obj.x</code> 读
备用构造	<code>@classmethod def from_row(cls, row):</code>
无状态工具	<code>@staticmethod def helper():</code>
数据类	<code>@dataclass</code> ; 可变默认值用 <code>field(default_factory=list)</code>
魔术方法	<code>__init__ __repr__ __eq__ __len__ __iter__ __enter__ __exit__</code>
继承检查	<code>isinstance(x, C) / issubclass(D, C) / C.__mro__</code>

A.5 模块与导入

写法	
导入模块	<code>import m</code> → <code>m.f()</code>
导入名字	<code>from m import f, g</code> → <code>f()</code>
别名	<code>import tkinter as tk</code> / <code>from x import y as z</code>
相对导入	<code>from . import x</code> / <code>from .sub import y</code>
脚本守卫	<code>if __name__ == "__main__":</code>

需求	写法
延迟导入	把 <code>import</code> 放进函数体（省启动时间）
加搜索路径	<code>sys.path.insert(0, str(Path(__file__).resolve().parent.parent))</code>
打包路径	<code>Path(sys.executable).parent if getattr(sys, "frozen", False) else Path(__file__).parent</code>

A.6 常用标准库

需求	写法
路径	<code>Path(a) / "b" / .parent.name.stem.suffix.resolve().exists()</code>
建目录	<code>p.mkdir(parents=True, exist_ok=True)</code>
读文本	<code>p.read_text(encoding="utf-8")</code>
保行尾写回	<code>p.write_bytes(text.encode("utf-8"))</code>
当前时间	<code>datetime.now() / date.today()</code>
时间格式化	<code>dt.strftime("%Y-%m-%d %H:%M") → "2026-09-21 14:05"</code>
日期运算	<code>d + timedelta(days=1)</code> ；月末天数 <code>calendar.monthrange(y, m)[1]</code>
JSON	<code>json.dumps(o, ensure_ascii=False) / json.loads(s)</code>
SQLite	<code>sqlite3.connect(p) + conn.row_factory = sqlite3.Row</code>
日志	<code>logging.getLogger(__name__) → logger.exception("...")</code>
子进程	<code>subprocess.run(..., capture_output=True, text=True, timeout=15)</code>
环境变量	<code>os.environ.get("NAME", "")</code>
正则	<code>re.compile(r"...") → .finditer(text) / .search / .sub</code>
哈希	<code>hashlib.sha256(b).hexdigest()</code>

A.7 高频惯用法

目的	写法
计频	<code>d[k] = d.get(k, 0) + 1</code>
设默认不覆盖	<code>d.setdefault(k, v)</code>
兜底默认（小心吞掉 0）	<code>x = value or DEFAULT</code>
提取一列	<code>[row["name"] for row in rows]</code>
过滤	<code>[x for x in xs if 条件]</code>
建索引	<code>{x.id: x for x in xs}</code>

目的	写法
去重保序	<code>list(dict.fromkeys(xs))</code>
多条件排序	<code>sorted(xs, key=lambda x: (-x.score, x.name))</code>
稳定二级排序	<code>sorted(xs, key=lambda x: (x.a, x.id))</code>
并列遍历	<code>for a, b in zip(xs, ys):</code>
带下标遍历	<code>for i, x in enumerate(xs, 1):</code>
交换	<code>a, b = b, a</code>
安全取值	<code>d.get(k, "")</code> 而不是 <code>d[k]</code>
幂等追加	追加前先 <code>find</code> 标记，命中就跳过
判 None	<code>if x is None:</code> (不要用 <code>== None</code>)
空值判真	<code>if not x:</code> 会连 <code>0 / "" / []</code> 一起匹配

附录 B · 模块清单

下表是生成这份手册时对源码目录**现算**的结果，顺序按文件行数从大到小。职责取自各模块自己的 docstring 首句。

模块	行数	类	函数	职责
main.py	5444	9	39	ExpiryManager - 个人系统（到期管理器）v2.0
tools_page.py	4485	6	13	工具包页面（重写版：分类+包管理、相对路径、图标抽取）
todo_page.py	2165	3	3	待办事项模块 - 界面层（View Layer）
study_notes_window.py	1844	3	3	学习笔记主窗口
embedded_admin_tools/api_demo_window.py	1607	1	0	（无模块 docstring）
scripts/test_tools_launcher.py	1576	0	36	系统工具箱「启动器」回归测试。
todo_db.py	1545	5	10	待办事项模块 - 数据层（Data Layer）
embedded_admin_tools/market_quote_window.py	1458	1	0	（无模块 docstring）
scripts/test_todo_ui.py	1355	1	29	待办页面 UI 回归测试：真实窗口 + 样例数据 + 逐项断言。
adb_page.py	1080	1	0	ADB 工具箱
scripts/test_todo.py	976	0	17	待办 / 提醒事项模块回归测试（数据层 + 节假日引擎，纯逻辑、不碰 Tk）。
embedded_admin_tools/error_code_service.py	929	1	2	错误码服务 - returnCode.json 内嵌版本
study_demo_window.py	925	2	0	Python 学习辅助模块
study_notes_db.py	895	4	2	学习笔记模块 - 数据层（Data Layer）
scripts/manual_engine.py	890	3	14	Python 语法手册的排版引擎 —— 把「块列表」渲染成一份带目录、页码、书签的 PDF。
qa_work_log_page.py	774	5	0	Q&A 问答 + 每周工作纪要 + 练习记录 页面
tools_db.py	768	1	38	工具库数据库
qa_work_log_db.py	707	8	4	Q&A 问答 + 每周工作纪要 + 练习记录 数据层
qa_extras.py	668	4	4	Q&A + 工作纪要 + 练习记录 扩展功能

模块	行数	类	函数	职责
<code>ui_theme.py</code>	592	3	0	全局 UI 主题定义。
<code>embedded_admin_tools/crypto_window.py</code>	573	1	0	(无模块 docstring)
<code>account_windows.py</code>	542	2	1	共享账户管理相关窗口。
<code>embedded_admin_tools/interface_service.py</code>	527	1	2	接口服务 - <code>interfaces.json</code> 内嵌版本
<code>markdown_view.py</code>	509	0	5	Markdown 渲染 (Typora 风格)
<code>expiry_dialogs.py</code>	492	3	2	到期管理相关弹窗。
<code>ui_components.py</code>	482	2	10	统一 UI 组件层。
<code>embedded_admin_tools/login_checker_window.py</code>	464	1	0	(无模块 docstring)
<code>baidu_disk_window.py</code>	463	2	0	百度网盘资料库管理窗口
<code>scripts/make_python_manual.py</code>	419	0	11	生成《Python 语法手册（配 ServerTimeDemo 项目实例）》PDF。
<code>tools_launcher.py</code>	408	0	23	<code>tools_launcher.py</code> —— 工具启动器的纯逻辑层。
<code>embedded_admin_tools/app_icon_service.py</code>	398	0	1	主程序图标内嵌服务
<code>console_page.py</code>	397	1	0	<code>console_page.py</code> - 控制台页面(日志输出 + 命令执行 + 工具维护)
<code>todo_icons.py</code>	375	0	47	待办模块的自绘图形资产。
<code>app_icons.py</code>	373	1	23	应用图形资产：应用标记 + 导航线性图标。
<code>scripts/project_audit.py</code>	367	1	10	项目静态审计脚本 — 只读分析，不修改任何被分析文件。
<code>scripts/prune_unused_imports.py</code>	343	0	10	未使用导入裁剪器 —— 依据 <code>pyflakes</code> 结果精确删除导入名，只读预览为默认。
<code>nav_sidebar.py</code>	336	1	0	左侧分组导航侧栏（自绘）。
<code>scripts/test_notes_markdown.py</code>	323	0	6	笔记页排版回归测试：Markdown 预览 tag + 编辑器外框 + 底部预览面板。
<code>scripts/test_notes_editor.py</code>	304	1	3	笔记编辑器回归测试：格式工具栏动作 + 源码淡化 + 语法速查 + 两处预览渲染。
<code>scripts/test_detail_dialog.py</code>	288	0	7	资源详情弹窗回归测试。
<code>app_version.py</code>	285	0	3	版本号与更新日志的单一数据源。
<code>scripts/make_app_icon.py</code>	285	0	10	生成应用图标 (<code>app.ico</code>)，并同步更新内嵌 <code>base64</code> 副本。
<code>scripts/test_expiry_table.py</code>	277	0	4	到期表格「单元格收敛」回归测试。

模块	行数	类	函数	职责
<code>study_demo_db.py</code>	271	0	20	Python 学习模块数据库层
<code>backend_page.py</code>	244	1	0	后台首页页面类。
<code>scripts/check_name_scope.py</code>	244	0	7	名字与作用域检查 —— 只读分析，不修改任何被检查文件。
<code>scripts/test_shell_ui.py</code>	242	0	3	外壳回归测试：自绘侧栏 + 顶栏 + 主题 token 的运行时验证。
<code>credential_process_dialogs.py</code>	233	3	0	账号中心与流程中心相关弹窗。
<code>scripts/scan_source_writers.py</code>	215	0	3	源码写入扫描 —— 找出会「就地重写源码」的脚本（只读分析，绝不修改文件）。
<code>scripts/runtime_smoke.py</code>	214	0	2	运行时冒烟测试 —— 真实执行关键代码路径，不依赖静态推断。
<code>embedded_admin_tools/login_memory_service.py</code>	212	2	1	（无模块 docstring）
<code>scripts/test_release_meta.py</code>	210	0	7	发布元信息回归：版本号 / 更新日志 / CHANGELOG.md 是否一致。
<code>scripts/test_log_setup.py</code>	203	0	2	log_setup 回归测试 —— 真实写文件、真实读回，逐条断言。
<code>credentials_page.py</code>	195	1	0	账号中心页面类。
<code>expiry_page.py</code>	185	1	5	到期管理页面类。
<code>log_setup.py</code>	176	0	5	全局日志配置 —— 在程序入口调用一次，全项目模块自动生效。
<code>page_components.py</code>	176	0	10	页面级组件层。
<code>dialog_form_style.py</code>	167	0	7	弹窗表单统一样式。
<code>process_page.py</code>	152	1	0	流程中心页面类。
<code>scripts/check_exe.py</code>	147	0	2	打包产物体检：exe 时间戳 / 体积 + 从归档里反向核对源码真的进去了。
<code>scripts/window_shot.py</code>	139	1	3	把 Windows 窗口抓成 PNG —— 用 PrintWindow，不依赖桌面是否可见。
<code>scripts/dep_inventory.py</code>	137	0	2	依赖清单盘点 —— 静态提取全项目 import，分类为 标准库 / 第三方 / 项目内。
<code>scripts/run_all_tests.py</code>	126	0	2	一条命令跑完全部回归套件，并汇总成一张表。
<code>scripts/verify_icon_generators.py</code>	113	0	3	图标生成器等价性验证 —— 只读比对，不写入任何目标文件。
<code>scripts/gen_app_icon_service.py</code>	69	0	2	由 app.ico 生成内嵌图标的服务模块。

模块	行数	类	函数	职责
scripts/gen_app_icon.py	61	0	2	生成主程序图标 app.ico（简化版占位图）。
scripts/gen_icon_service.py	61	0	1	由 base64 文本生成内嵌图标的服务模块。
embedded_admin_tools/api_client.py	55	1	0	（无模块 docstring）
scripts/gen_changelog.py	50	0	2	从 app_version.VERSION_HISTORY 生成仓库根目录的 CHANGELOG.md。
embedded_admin_tools/crypto_service.py	22	1	0	（无模块 docstring）
embedded_admin_tools/error_code_service.py	7	0	0	兼容包装：统一转发到 embedded_admin_tools.services.error_code_service。
embedded_admin_tools/interface_service.py	7	0	0	兼容包装：统一转发到 embedded_admin_tools.services.interface_service。

B.1 数据目录结构

1	ExpiryManager_Data/	← 全部运行期数据都在这一个目录里
2	—— expiry_manager.db	主数据库（23 张表）
3	—— login_memory.json	登录记忆（含密码密文）
4	—— logs/app.log	轮转日志（1 MB × 3 份）
5	—— account_images/	账号中心上传的图片
6	—— study_notes_images/	笔记图片
7	—— study_notes_attachments/	笔记附件
8	—— process_flow_images/	流程图图片
9	—— ...	导出文件、历史记录等

▲ 目录名与迁移映射见 main.py 的 _MIGRATION_MAP

所有数据放一个目录，是给「备份」用的

一个程序的数据散落在十几个地方时，「备份」就变成了需要文档的操作。收进一个目录之后，用户只需要「拷走这个文件夹」。

配套的设计是 `_MIGRATION_MAP + .migrated` **标记文件**：首次启动时把散落在程序目录里的老文件搬进 `ExpiryManager_Data/` 并留下标记，之后不再执行。**迁移只做一次、且可重复检测**，这也是幂等思想的一个应用。

B.2 测试套件与基线

项目共 11 个回归套件，下表是本次生成时的实测结果（跑一遍现统计）：

套件	通过	失败
test_notes_markdown	60	0
test_notes_editor	55	0
test_shell_ui	35	0

套件	通过	失败
test_tools_launcher	230	0
test_log_setup	17	0
test_expiry_table	23	0
test_detail_dialog	35	0
test_release_meta	82	0
test_todo	247	0
test_todo_ui	251	0
runtime_smoke	18	0
合计	1053	0

附录 C · 坑对照表

这份附录把前面各处散落的「不要这么写」集中成一张表，方便当检查清单用。左边是**错误写法**，中间是**症状**，右边是**正确写法**。

C.1 语法与语义类

错误写法	症状	正确写法
<code>def f(x, items=[])</code>	所有调用共享同一个列表，越追加越长	<code>items=None + 函数内 if items is None: items = []</code>
<code>@dataclass</code> 里 <code>tags: list = []</code>	同上，且更难发现	<code>field(default_factory=list)</code>
<code>for x in items:</code> <code>items.remove(x)</code>	元素被跳过，删不干净	遍历 <code>list(items)</code> 副本，或重建列表
<code>for k in d: del d[k]</code>	<code>RuntimeError: dictionary changed size</code>	先 <code>for k in list(d):</code>
循环里 <code>lambda: f(i)</code>	所有回调都用最后的 <code>i</code>	<code>lambda i=i: f(i)</code> ，或用工厂方法
<code>x == None</code>	自定义 <code>__eq__</code> 时结果可能不符合预期	<code>x is None</code>
<code>if not x:</code> 判数字	<code>0</code> 被当成「没有值」	<code>if x is None:</code>
<code>except:</code>	连 <code>Ctrl+C</code> 都吞掉，程序无法中断	<code>except Exception:</code>
<code>assert</code> 做输入校验	<code>python -O</code> 时校验整段消失	<code>if not ok: raise ValueError(...)</code>
<code>except Exception: pass</code>	功能长期不工作而无人知晓	至少记一条日志；或注明降级后的状态
<code>d["k"]</code> 读可能没有的键	<code>KeyError</code>	<code>d.get("k", 默认值)</code>
<code>int("")</code>	<code>ValueError</code>	<code>int(x or 0)</code> ，或包 <code>try</code>
<code>0.1 + 0.2 == 0.3</code>	<code>False</code>	<code>abs(a - b) < 1e-9</code> ；金额用 <code>int</code> 或 <code>Decimal</code>
<code>round(2.675, 2)</code>	得 <code>2.67</code>	<code>Decimal.quantize(..., ROUND_HALF_UP)</code>
<code>strptime</code> 解析 <code>9:05</code>	部分实现解析不出非零填充小时	手工 <code>int()</code> 拆（见 <code>parse_moment</code> ）
<code>strftime("%a")</code> 判星期	中文系统上输出中文	<code>date.weekday()</code> 数值判断

C.2 文件与编码类

错误写法	症状	正确写法
<code>Path.write_text(...)</code> 改源码	整个文件的行尾被翻成 CRLF, diff 爆炸	<code>read_bytes + write_bytes</code>
<code>open(p, "w")</code> 不写编码	中文在 Windows 上乱码或报错	<code>encoding="utf-8"</code>
CSV 导出用 <code>open(p, "w")</code>	每行多一个空行	<code>newline=""</code> (+ <code>utf-8-sig</code> 给 Excel)
程序内部读写用 <code>utf-8-sig</code>	JSON 解析失败 (BOM)	内部一律 <code>utf-8</code>
<code>__file__</code> 定位数据目录	打包后数据写进临时目录, 关掉就丢	<code>sys.frozen + sys.executable</code>
一次替换多个锚点不校验	锚点写错时静默改错地方	每处锚点断言恰好命中一次, 否则整体拒写
补丁脚本用 <code>findall</code> 取多分组	拿到的是元组列表, <code>hits[0]</code> 不是字符串	用 <code>finditer</code> 并按 <code>group(n)</code> 取
脚本里写转义引号	反斜杠被链路吃掉, 外层字符串提前闭合	内容一律用单引号 / 三单引号
全量替换不做行数复核	行尾被改动而看不出来	<code>git diff --numstat</code> 核对变更行数

C.3 GUI 类

错误写法	症状	正确写法
<code>command=self.save()</code>	绑定时就执行一次, 回调变成 <code>None</code>	<code>command=self.save</code>
<code>def handler(self):</code> 配 <code>bind</code>	<code>TypeError</code> : 少一个位置参数	<code>def handler(self, event=None):</code>
先 <code>pack expand=True</code> 的控件	后面 <code>pack</code> 的控件被饿死, 完全不显示	先 <code>pack</code> 固定尺寸控件, 最后 <code>pack</code> 内容区
滚动条后于滚动区 <code>pack</code>	滚动条被挤出可视区	滚动条先 <code>pack</code>
子类 <code>__init__</code> 里先干别的	<code>TclError</code> : <code>self</code> 还不是控件	第一句 <code>super().__init__(...)</code>
主窗口 <code>withdraw()</code> 后查 <code>ismapped()</code>	所有控件都返回 0, 误判成「被饿死」	先 <code>deiconify()</code> 再判定
布局前查 <code>winfo_width()</code>	得到 1 (还没算)	先 <code>update_idletasks()</code> / <code>update()</code>
缓存 Tk 对象不探活	解释器换过后 <code>TclError</code> : <code>invalid command name</code>	用前调一个无害方法探活, 失败就清缓存
销毁后继续用控件	同上	<code>if w.winfo_exists(): w.destroy()</code>
回调里不 <code>try</code>	异常被 Tk 事件循环吞掉, 界面静默失灵	回调内部 <code>try/except</code> + 记日志
用线程更新界面	随机崩溃, 极难复现	用 <code>after()</code> 回到主线程
定时器只在成功路径续期	一次异常后定时器永久死掉	<code>finally</code> : 里续期

错误写法	症状	正确写法
退出时不取消定时器	回调打到正在销毁的控件上	置退出标志，回调先检查它

C.4 数据与打包类

错误写法	症状	正确写法
f-string 拼 SQL	SQL 注入；名字带单引号直接语法错误	? 占位符 + 参数元组
循环里逐条 commit	慢（每条一次磁盘刷写）	executemany + 只 commit 一次
不设 row_factory	只能按下标取值，加一列就让取值全错位	conn.row_factory = sqlite3.Row
不设 PRAGMA foreign_keys	外键形同虚设	每个连接显式 PRAGMA foreign_keys = ON
ALTER TABLE 前不查列	重复启动报「duplicate column」	先 PRAGMA table_info 再按需补
新列不给默认值	老数据行没有合法值	ADD COLUMN ... NOT NULL DEFAULT ''
只按键排序	顺序随内部存储变化，看起来会「自己变」	加第二级 id ASC
改结构时删列	老用户数据丢失	加列永远只做加法
打包脚本先清空 dist	用户运行期数据永久丢失	只删制品，数据另存另拷
只信打包日志	漏模块只在运行版炸	打开归档 反向核对 符号
pip freeze 覆盖依赖锁	把间接依赖锁成当前版本，难以复现	只锁直接依赖

附录 D · 工具脚本与开发流程

项目跑到后来长出了一套自己的开发工具，都在 `scripts/` 目录下。它们既是「本项目怎么维护」的说明，也是写 Python 工具脚本的现成参考。

脚本	作用	值得学的地方
<code>run_all_tests.py</code>	一条命令跑完全部套件并汇总成表	开场先探 <code>tkinter</code> 是否可用，解释器不对时直接说该换哪个，不闷头跑出一片假红
<code>window_shot.py</code>	把窗口抓成 PNG	<code>PrintWindow + PW_RENDERFULLCONTENT</code> ；抓完校验颜色数，只抓到一种就抛错
<code>check_exe.py</code>	打包后反向核对归档里的符号	递归遍历 <code>code object</code> 的 <code>co_names / co_consts</code>
<code>test_*.py</code> (11 个)	回归测试	每个都用 <code>check(名称, 条件, 现场值)</code> 的写法，失败时打印真实值
<code>gen_changelog.py</code>	从 <code>app_version.py</code> 生成 CHANGELOG	单一数据源：版本号只有一处，日志自动重建
<code>project_audit.py</code>	AST 静态审计	函数长度、嵌套深度、重复代码
<code>check_name_scope.py</code>	补 <code>pyflakes</code> 盲区	<code>except</code> 名字作用域、闭包作用域
<code>prune_unused_imports.py</code>	批量删未使用导入	字节级改写 + 语法安全校验
<code>dep_inventory.py</code>	依赖清单	统计模块级 <code>import</code> 的第三方包
<code>make_python_manual.py</code>	生成本手册	正文 (<code>manual_content_*.py</code>) 与排版 (<code>manual_engine.py</code>) 分离，统计数字全部现算

D.1 这本手册自己怎么生成

手册的正文与排版是分开的：文字写在 `manual_content_a.py` 与 `manual_content_b.py` 里（一长串「块」），排版规则在 `manual_engine.py`（字体、代码着色、目录、书签、页眉页脚），`make_python_manual.py` 把两边接起来。改文字不会碰到版式，改版式也不会碰到文字。

```
1 # 出三册（默认行为）
2 python scripts/make_python_manual.py
3
4 # 只要一本合订本
5 python scripts/make_python_manual.py --single
6
7 # 换输出目录
8 python scripts/make_python_manual.py --outdir build/manual
```

▲ 默认出三册；`--single` 出合订本；`--outdir` 换输出目录。命令都在项目根目录执行。

生成入口里有两条硬约束，都是踩过坑之后加的：

- **一级标题决定册次。**三册的边界写在 `make_python_manual.py` 的 `VOLUMES` 里，按一级标题匹配、不按页码——内容一改页码必然变，标题是稳的。匹配不到、或一个标题同时命中两册，脚本立刻退出，不允许静默串册。
- **统计数字一律现算。**文件数、行数、数据表数、回归测试的通过项数，都在生成时扫源码、读测试结果得到，正文里只留一个短记号（形如 `n_root`），由生成器填进去。

为什么一个数字都不许写死

手写的数字没有保质期：数据库加一张表，正文里的「24 张表」就错了，而且**没人会回头改它**——错的那个数字会一直印在纸上。

所以凡是机器能数出来的，就不要人来数。

D.2 本项目自己的「补丁纪律」

前面 13.1 与 13.2 讲的是行尾，这里把它拼成一套完整的操作流程——这是本项目这几轮迭代里最核心的一条工程经验。

1. **写一个补丁脚本，而不是手工改文件。**脚本里每处替换都带断言：锚点必须**恰好命中一次**，否则整体拒写。这一步拦住的是「替换到了别的地方」和「文件已经被改过」。
2. **顺序化执行，不要在同一个文件上并行编辑。**同一文件的两处修改若并行提交，后一次的写入会基于旧内容，**静默丢掉前一次的改动**。
3. **字节读、字节写**（`read_bytes / write_bytes`），中途 `decode` 处理文本、`encode` 写回，全程不碰文本模式的行尾翻译。
4. **先记下当前的行尾指纹：**纯 CRLF 的文件，断言 `count(b"\r\n") == count(b"\n")`；纯 LF 的文件，断言 `count(b"\r\n") == 0`；混排的文件（`main.py` 有 167 个单独 LF），断言这个数量没变。
5. **写完立刻** `py_compile.compile(path, doraise=True)`，语法错误当场暴露。
6. **收尾用** `git diff --numstat` **核对：**变更行数与预期不符，说明动到了不该动的地方，`git checkout --` 回退重来。

这套流程里最容易被跳过、也最危险的一条

「删除」最危险，仅次于它的就是「全量替换」。

本项目有一条硬规则：**不用** `git rm <目录>/<文件>`。在某个 Git 版本下，只要路径里带 `/`，`git rm` 会删掉整个顶层目录树——曾经因此一次性误删 25 个文件。正确做法是 `rm <file>` 再用 `git add -A` 让 Git 记录，而且**删除之前先提交**，这样任何误删都能 `git checkout --` 完整恢复。

D.3 这套流程的通用形状

把上面六步抽象出来，它与具体的项目、语言都无关：

任何「程序化改写文件」都适用的四条

- ① **先验证前提**（锚点唯一、现状符合预期），再动手；
- ② **宁可整体失败，不要部分成功**（一处不对劲就什么都不写）；
- ③ **写完立刻做一次独立校验**（编译、行数、差异）；
- ④ **保证可回退**（先提交，再改）。

这四条合起来就是一个最小可用的「安全批量改写」协议。

D.4 待办（项目的下一步）

以下几条写在项目自己的记忆文件里，属于已经排定但还没做的：

- **拆 `main.py`**（当前 5,400 余行）—— 入口、主窗口、通用工具函数三者分离。
- **`todo_db.py` 的迁移逻辑表驱动化** + SQL 标识符白名单。
- **Database 的 Mixin 拆分** —— 现在 23 张表的操作都在一个类里。
- **引入 `ruff` 与 `mypy`**，把静态检查从「只看告警数」推进到「有规范标准」。
- **其余页面的空状态** —— 待办模块的空状态已经做完，其它页面还没有。

结语

这本手册从头到尾只做了一件事：**把语法点钉在真实代码上**。如果你读完之后记住的不是「Python 有推导式和装饰器」，而是「定时器续期要放 `finally`」「改源码要走字节」「`field(default_factory=list)` 是为什么」——那它就起作用了。

附录里的模块清单、体量统计、测试基线都是**生成时现算的**，所以只要重新跑一次生成脚本，它们永远与仓库同步；而正文里的代码是手抄的，改动后需要人工同步。这一点也提醒了一件更普适的事：**能自动化的东西就让机器算，手工抄写的内容一定要标注它的保质期**。